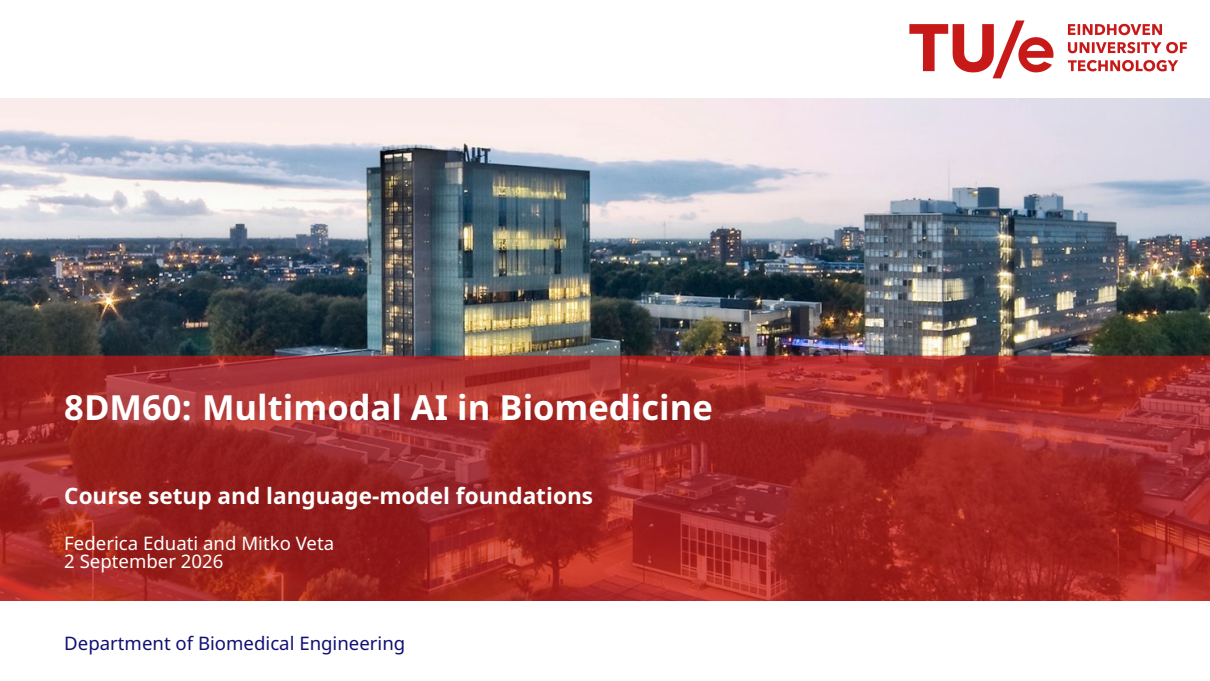


An aerial photograph of the TU/e campus in Eindhoven at dusk. The image shows several modern buildings with glass facades, some of which are illuminated from within, casting a warm glow. The sky is a mix of soft pinks, purples, and blues. In the foreground, there are trees and a parking lot. The overall scene is a mix of urban architecture and natural elements.

8DM60: Multimodal AI in Biomedicine

Course setup and language-model foundations

Federica Eduati and Mitko Veta
2 September 2026

Course outline

Lectures and guided self-study schedule

Week	Lecture	Guided self-study
1	Introduction and language models	Introduction to project work
2	Vision models	Language models
3	Omics models	Vision models
4	Models for hospital data	Omics models
5	Integration of AI in clinical workflow	Models for hospital data
6	Agentic AI	Integration of AI in clinical workflow
7	<i>Developing AI products for healthcare*</i>	Agentic AI

* Guest-lecture content is not assessed in the written examination.

Lectures and guided self-study have different goals

Session	Purpose
Lecture 1	Classical lecture introducing the course and establishing language-model and Transformer foundations.
Lectures 2–6	Flipped-classroom sessions: prepare beforehand; use class to clarify concepts, compare methods, examine trade-offs, and evaluate evidence.
Lecture 7	Guest lecture on developing AI products for healthcare.
Guided self-study	Problem-solving practice: reason individually, discuss approaches, and apply lecture concepts to unfamiliar cases.

Guided self-study is practice for the written examination

1. Work through approximately four or five open, problem-based questions.
2. Think individually for about 10 minutes, then discuss each problem in class.
3. Apply lecture concepts to an unfamiliar method or biomedical AI system.

Questions may build on the lecture papers or introduce a related method that requires the same reasoning.

How to learn in this course

Before class

From Lecture 2, identify each paper's problem, central method, training objective, key evidence, and main limitation.

In class

Bring notes and questions. Actively engage in the discussion with peers and the instructor.

In project work

Groups of five use the starter notebook to define a focused medical imaging research question and submit a one-page poster plus reproducible code.

The project has two Canvas submission points

1. Define research topic/question

Describe the research topic or question you want to address in **up to 400 words**.

- Optional figures and references do not count toward the word limit.
- Upload one .doc, .pdf, or .md file.
- **0 points**
- **Due: 11 September 2026, 23:59**

2. Project work

Submit **one ZIP archive** containing:

- the one-page poster in PDF format; and
- documented code that can fully reproduce the experiments.
- **15 points**
- **Due: 14 October 2026, 23:59**

Both assignments are available to everyone and must be submitted through Canvas.

Assessment: project work and written examination

Component	Weight
Project deliverables: poster and code	15%
Group-specific project exam questions	15%
Multiple-choice exam questions	20%
General open/problem-based exam questions	50%

- **Written examination:** 28 October 2026, 13:30–16:30.
- **Resit examination:** 20 January 2027, 18:00–21:00.

From machine learning to language models

Learning objectives

By the end of this lecture, you should be able to:

- Explain how data and output structure motivate different machine-learning architectures.
- Describe translation as conditional sequence generation and distinguish encoder, decoder, and decoder-only models.
- Explain autoregressive language modelling and how raw text supplies self-supervised next-token targets.
- Trace text through tokenization, embeddings, position information, attention, contextual representations, and output probabilities.
- Distinguish parallel training from sequential generation and discuss what next-token prediction does and does not guarantee.

What do machine-learning problems have in common?

$$y = f(x)$$

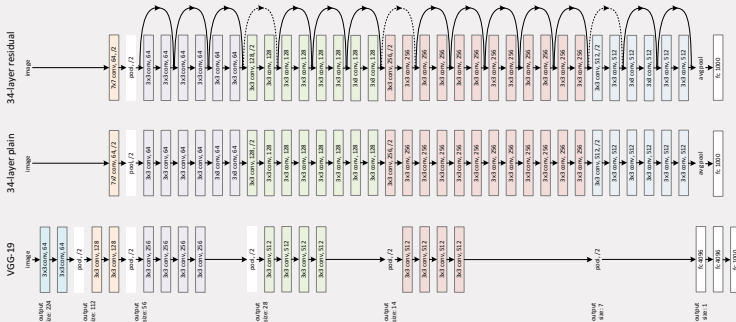
Input x	Output y	Task
Patient characteristics	Disease category	Classification
Histopathology image	Tumour subtype	Classification
MRI image	Organ mask	Segmentation
CT image	Survival risk	Regression

In supervised learning, examples $\{(x_i, y_i)\}_{i=1}^n$ are used to learn a function f .

Learned functions take many model forms

Possible model families

- Linear regression
- Logistic regression
- Decision trees
- Random forests
- Neural networks

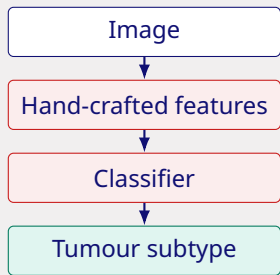


He et al., *Deep Residual Learning for Image Recognition*, Figure 3.

A neural network can represent a deep composition of learned transformations.

Neural networks learn intermediate representations

Traditional machine learning

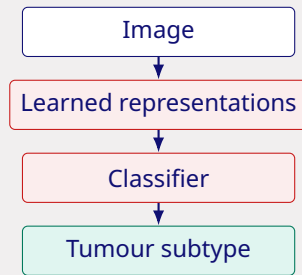


Features are designed before classifier training.

Deep networks learn task-useful representations directly from data.

VS.

Deep learning



Representations and the classifier are learned jointly.

Data structure should guide the architecture choice

Patient table

Histopathology image

Molecular graph

Clinical text

Which architecture would you choose for each input?

Base the choice on the interactions the model should represent efficiently.

Architecture should match the structure of the data

Data	Relevant structure	Architecture
Patient table	Fixed-length features	Multilayer perceptron (MLP)
Histopathology image	Spatial locality and repeated patterns	Convolutional network (CNN)
Molecular graph	Relations defined by edges	Graph neural network (GNN)
Clinical text	Order and contextual dependencies	Transformer

Inductive biases build useful assumptions about the data into the architecture.

Language supports several machine-learning tasks

Task	Input	Output
Document classification	Clinical note	One diagnosis or category
Token classification / segmentation	Clinical note	A label or span for each relevant token
Sequence generation	Text or another modality	A variable-length text sequence

The architecture should match the output structure: one label, labels over a sequence, or a newly generated sequence.

Why is translation different from classification?

Classification predicts a label

Input: histopathology image

Output: invasive ductal
carcinoma

Encoded label: 1

Translation predicts a sequence

Input: The patient has pneumonia.

Output: De patiënt heeft een
longontsteking.

Classification returns one label; translation conditions on one sequence and generates another whose length is not fixed.

Many language tasks map one sequence to another

Task	Input sequence	Output sequence
Translation	Sentence in one language	Sentence in another language
Summarization	Long document	Short summary
Grammar correction	Uncorrected sentence	Corrected sentence
Report generation	Clinical findings or observations	Written report
Text simplification	Complex text	Simplified text

The shared challenge is to represent the input sequence and generate a new output sequence.

Sequence generation starts by representing the input

Before a model can generate an output sequence, it must turn the input sentence into numerical, context-dependent representations.

Token

One text unit processed by the model.
For now, think of a word in an ordered sentence.

The patient has pneumonia

Contextual representation

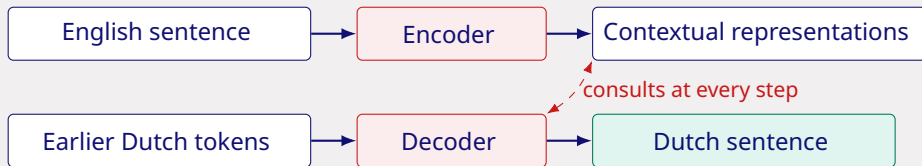
A numerical vector for one token that also summarizes relevant information from the surrounding sentence.

The representation of **patient** can encode information from **has pneumonia**.

The encoder produces one context-dependent representation for every input token.

Tokens and tokenization are discussed in more detail later in this lecture.

The encoder represents; the decoder generates



Encoder

- Reads the complete English sentence.
- Produces one contextual representation per input position (in essence, for each input token).

Decoder

- Uses the source representations and earlier Dutch tokens.
- Predicts the next output token.

This encoder-decoder setting motivated the original Transformer for machine translation.

The decoder generates tokens autoregressively

The translation is constructed incrementally:

De

$$P(y_t \mid y_{<t}, x)$$

- x : source sentence
- $y_{<t}$: previous output tokens
- y_t : next output token

Autoregressive generation feeds each selected token back into the prefix and continues until an end-of-sequence token is selected.

The decoder generates tokens autoregressively

The translation is constructed incrementally:

De

De patiënt

$$P(y_t \mid y_{<t}, x)$$

- x : source sentence
- $y_{<t}$: previous output tokens
- y_t : next output token

Autoregressive generation feeds each selected token back into the prefix and continues until an end-of-sequence token is selected.

The decoder generates tokens autoregressively

The translation is constructed incrementally:

De

De patiënt

De patiënt heeft

$$P(y_t \mid y_{<t}, x)$$

- x : source sentence
- $y_{<t}$: previous output tokens
- y_t : next output token

Autoregressive generation feeds each selected token back into the prefix and continues until an end-of-sequence token is selected.

The decoder generates tokens autoregressively

The translation is constructed incrementally:

$$P(y_t \mid y_{<t}, x)$$

De

De patiënt

De patiënt heeft

De patiënt heeft een

- x : source sentence
- $y_{<t}$: previous output tokens
- y_t : next output token

Autoregressive generation feeds each selected token back into the prefix and continues until an end-of-sequence token is selected.

The decoder generates tokens autoregressively

The translation is constructed incrementally:

$$P(y_t | y_{<t}, x)$$

De

De patiënt

De patiënt heeft

De patiënt heeft een

De patiënt heeft een longontsteking <EOS>

- x : source sentence
- $y_{<t}$: previous output tokens
- y_t : next output token

Autoregressive generation feeds each selected token back into the prefix and continues until an end-of-sequence token is selected.

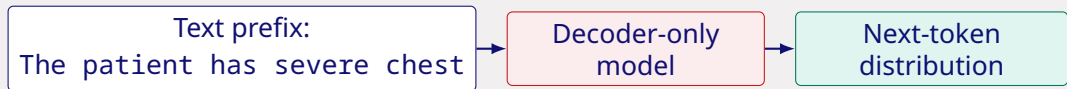
Language models assign probabilities to token sequences

A language model estimates how likely a token is given its context, and therefore assigns a probability to a sequence.

Model	Context	Probability
Translation decoder	Source sentence x and earlier target tokens $y_{<t}$	$P(y_t \mid y_{<t}, x)$
Decoder-only language model	Earlier tokens in one sequence $x_{<t}$	$P(x_t \mid x_{<t})$

Translation already contains conditional language modelling. Decoder-only pretraining removes the separate source sequence and learns directly from raw text.

A decoder-only language model removes the encoder



$$P(x_t | x_{<t})$$

The model predicts the next token from the tokens that precede it. Repeating this operation generates a continuation.

One sequence supplies many training examples

Input	Target
The	patient
The patient	has
The patient has	severe
The patient has severe	chest
The patient has severe chest	pain

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_{<t})$$

Each factor is the probability assigned to the observed token x_t , given all earlier tokens $x_{<t}$.

The probability of a sequence is decomposed into a series of next-token predictions.

Raw text supplies its own next-token targets

Observed text

The biopsy showed invasive carcinoma

Derived training example

Input: The biopsy showed invasive
Target: carcinoma

The optimization remains supervised.

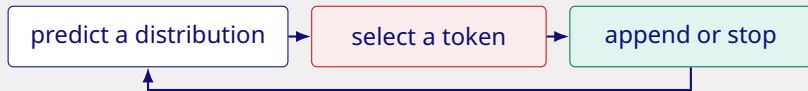
- Conventional supervised learning uses externally provided targets.
- Self-supervised learning derives targets automatically from observed data.

This makes learning from very large text collections possible without manual labels for every example.

Generation repeats the same next-token step

At inference time, each selected token is appended to the context and becomes input to the next prediction.

Step	Current context	Selected token	Action
1	The patient has severe chest	pain	Append
2	The patient has severe chest pain	.	Append
3	The patient has severe chest pain.	<eos>	Stop



Repeated fixed-vocabulary decisions produce a continuation whose content and length are not fixed in advance.

Tokens become indices, then embedding vectors

Representation steps

1. A vocabulary defines the available tokens.
2. The tokenizer splits text into those tokens.
3. Each token maps to an arbitrary integer ID.
4. That ID selects one learned row from an embedding table E .

Illustrative example

Text	severe chest pain
Tokens	[severe] [chest] [pain]
Token IDs	[4182, 927, 5102]
Lookup	$E[927] = [0.12, -0.44, \dots]$

IDs only locate rows; their numerical values do not encode similarity or meaning.

How can a tokenizer handle a long medical word?

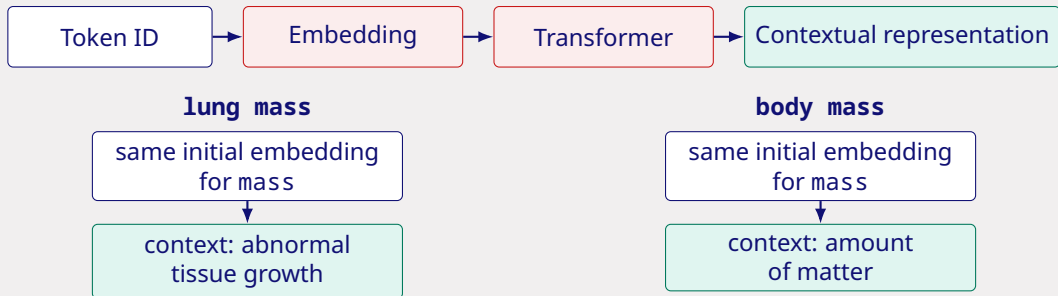
Text: immunohistochemistry

One possible split:



- Tokenizers can represent unfamiliar words by combining reusable word pieces.
- Different tokenizers may split immunohistochemistry differently.
- The resulting token IDs are arbitrary indices, not measurements of meaning.

Token embeddings do not contain complete contextual meaning



The embedding is fixed for the token; the Transformer produces a context-dependent representation.

Why not process language with a CNN or an RNN?

CNNs emphasize local patterns; language also requires direct access to distant context.

Recurrent neural networks

- Are designed for sequences.
- Support variable-length input.
- Require sequential computation.
- Are difficult to parallelize.
- Can struggle with long-range dependencies.

Transformers

- Support parallel training across sequence positions.
- Allow direct interaction between distant tokens.
- Scale more effectively with data and compute.

Transformer components solve distinct problems

Requirement	Transformer mechanism
Represent order	Positional information
Connect tokens and select relevant context	Self-attention
Ignore future tokens	Causal mask
Transform representations	Feed-forward network
Support stable deep training	Residual connections and normalization

Together, these mechanisms encode order, mix information across tokens, and make deep transformation trainable.

Transformers add position to token representations

$$h_i^{(0)} = e(\text{token}_i) + p_i$$

Token embedding $e(\text{token}_i)$

Represents which token occurs at position i .

The token `patient` starts with the same learned token embedding wherever it appears.

Position representation p_i

Represents where that token occurs in the sequence.

Common choices include learned absolute embeddings, sinusoidal encodings, and relative or rotary encodings.

Adding position information lets attention distinguish sequences such as “dog bites man” and “man bites dog.”

Sinusoidal encodings represent position at multiple scales

For model dimension $d = 8$, the original Transformer uses four sine-cosine pairs with progressively longer wavelengths:

$$p_i = \left[\sin(i), \cos(i), \sin\left(\frac{i}{10}\right), \cos\left(\frac{i}{10}\right), \sin\left(\frac{i}{100}\right), \cos\left(\frac{i}{100}\right), \sin\left(\frac{i}{1000}\right), \cos\left(\frac{i}{1000}\right) \right].$$

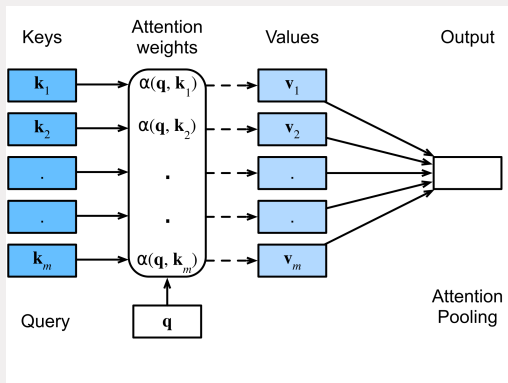
Example at position $i = 1$

$$p_1 \approx [0.841, 0.540, 0.100, 0.995, 0.010, 1.000, 0.001, 1.000]$$

- Early dimensions change rapidly with position.
- Later dimensions change slowly and encode coarser position differences.
- Every position receives a distinct, deterministic vector.

Source: Vaswani et al., *Attention Is All You Need*.

Attention matches a query to keys and mixes their values



Source: Zhang et al., *Dive into Deep Learning*, Fig. 11.1.1 (CC BY-SA 4.0).

One attention output

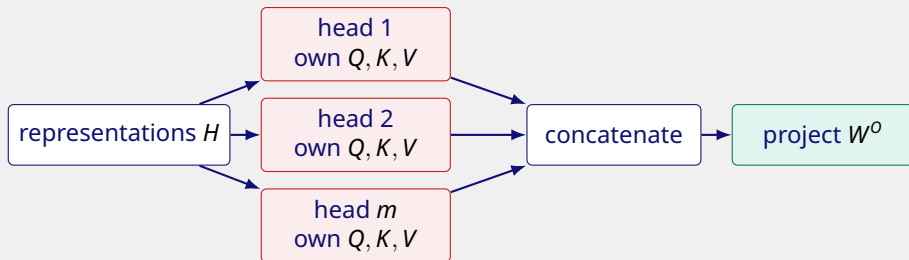
1. Compare the query q with every key k_i .
2. Normalize the scores into attention weights α_i .
3. Return a weighted sum of the corresponding values v_i .

Self-attention

Queries, keys, and values come from the same sequence. The next slide shows how several independently projected attention operations are combined.

Multi-head attention combines several learned views

$$\text{head}_j = \text{Attention}\left(HW_j^Q, HW_j^K, HW_j^V\right), \quad \text{MHA}(H) = \text{Concat}(\text{head}_1, \dots, \text{head}_m)W^O$$



Different heads can learn different interaction patterns, but no head is guaranteed to have one fixed semantic role.

Transformer blocks mix context and transform tokens

$H \in \mathbb{R}^{T \times d}$ contains the block input: one d -dimensional representation for each of T sequence positions.

$$H' = \text{Norm}(H + \text{Attention}(H)), \quad H'' = \text{Norm}(H' + \text{FFN}(H'))$$

Self-attention $\rightarrow H'$

Mixes information from other permitted positions into each token.

Feed-forward network $\rightarrow H''$

Applies the same nonlinear transformation at every position.

Residual connections

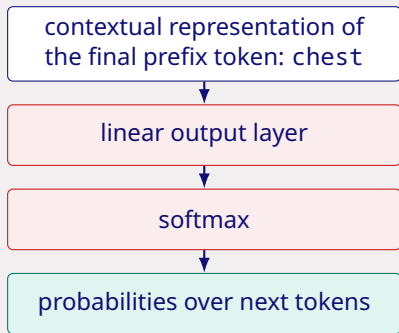
Add each sublayer input back to preserve information and gradient flow.

Normalization

Keeps activations well scaled across stacked blocks.

The last prefix representation predicts the next token

Prefix: The patient has severe chest



Candidate next tokens

pain	0.61
pressure	0.17
discomfort	0.11
pneumonia	0.04
other tokens	0.07

pain is the observed target; training increases the probability assigned to it.

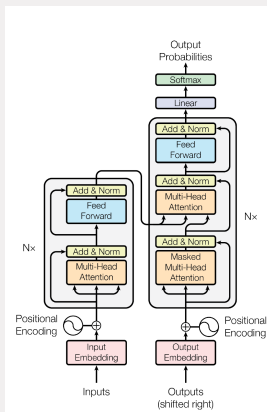
What prevents the model from cheating during training?

The complete sequence is available for efficient training, but a causal mask blocks every future position.

Representation at	The	patient	has	chest	pain
The	✓	×	×	×	×
patient	✓	✓	×	×	×
has	✓	✓	✓	×	×
chest	✓	✓	✓	✓	×
pain	✓	✓	✓	✓	✓

✓: visible context ×: hidden future token

The original Transformer uses two stacks



Encoder stack

Reads and contextualizes the complete source sequence.

Decoder stack

Generates the target from its existing prefix while consulting the encoded source.

Historical role

This architecture was designed for translation. Decoder-only LLMs omit the encoder and cross-attention.

Reproduced from Vaswani et al., *Attention Is All You Need*, NeurIPS 2017, Figure 1.

Attention pathways in translation

Attention pattern	Visible information	Role
Encoder self-attention	Complete source sentence	Contextualize every source token
Masked decoder self-attention	Earlier target tokens only	Represent the generated prefix without seeing its future
Encoder-decoder attention	Decoder queries; source keys and values	Select source information for the next target token

Decoder-only language models retain causally masked self-attention over the context and omit the other two attention pathways.

Training is parallel; generation is autoregressive

During training

1. Provide the complete sequence.
2. Predict the next token at every position.
3. Use a causal mask to hide future tokens.
4. Compute loss over all positions.

many next-token predictions
in parallel

During generation

1. Predict one token.
2. Append it to the sequence.
3. Process the extended context.
4. Repeat.

one new token
at a time

Next-token learning supports broad transfer

Accurate prediction can require

- grammar and meaning;
- factual and medical associations;
- document and discourse structure;
- dependencies across distant tokens.

At foundation-model scale

- representations can transfer to tasks not explicitly supervised;
- examples in the prompt can specify a new task;
- one model can support many tasks through different contexts and adaptation data.

Transfer is broad, but performance still depends on the task, context, data, and evaluation.

What does fluent next-token prediction fail to guarantee?

The objective does not guarantee:

- factual correctness;
- causal understanding;
- reliable reasoning;
- calibrated uncertainty.

Nor does it guarantee:

- faithful explanations;
- clinical safety;
- alignment with user intentions.

In medicine, linguistic plausibility is not evidence of clinical correctness.

Instruction tuning teaches response behaviour

Pretraining examples come from raw text

The patient has severe chest
→ predict pain

The base model learns broad continuation patterns from many sequences.

Instruction-tuning examples pair a request with a response

Instruction: Summarise this report.

Response: The radiograph shows...

Training still predicts next tokens, but the desired response supplies the targets.

Preference training can further favour responses judged more helpful or appropriate; it is a separate stage from instruction tuning.

Language-model design choices shape medical AI systems

Five independent choices

- **Representation:** what the model receives numerically
- **Architecture:** how sequence information is combined
- **Pretraining objective:** which broad patterns are learned
- **Adaptation data:** which responses are encouraged
- **Inference context:** what evidence is available now

Throughline: Transformers learn contextual sequence representations; adaptation and inference choices turn them into task-specific systems.

These foundations support

- Medical language models for question answering, extraction, and report generation
- Retrieval-augmented generation
- Vision-language and multimodal foundation models
- Instruction tuning for medical tasks
- Evaluation and safety

Adapting foundation models for biomedical applications

From a general pretrained model to a task-specific biomedical system

Learning objectives

By the end of this section, you should be able to:

- Compare prompting, retrieval-augmented generation, and fine-tuning as adaptation strategies.
- Explain why full fine-tuning is expensive and how parameter-efficient fine-tuning reduces trainable parameters.
- Interpret the LoRA parameterization and explain its rank, scaling, target modules, and inference options.
- Explain quantization and how QLoRA combines a quantized frozen base model with trainable LoRA adapters.
- Distinguish parameter adaptation from the training objective, including instruction tuning.

A foundation model is not yet a biomedical system

Pretraining provides

- broad language or multimodal representations;
- general continuation and generation ability;
- reusable parameters learned from large datasets.

A biomedical application still requires

- a precise task and intended use;
- appropriate domain data or external evidence;
- an adaptation and interaction strategy;
- evaluation in the intended population and workflow.

The system is the model plus its data, context, interface, evaluation, and deployment process.

Why not train a new foundation model?

Typical training scale

- Trillions of tokens
- Billions of parameters
- Thousands of GPUs
- Weeks to months of training

Typical pretraining corpora

- Web pages and books
- Scientific papers
- Code
- Wikipedia

Training a new foundation model at this scale is beyond the resources of almost every research group or hospital.

Adaptation can change context, evidence, or parameters

A prompt is the instruction, examples, data, and other context presented to the model for one request.

Strategy	What changes	Persistence
Prompting	Instructions, examples, supplied context	Per request
Retrieval-augmented generation	External passages added to the input	Per request
Fine-tuning	Model weights or added parameters	Persists after training

Choose the smallest intervention that addresses the actual limitation: context, missing evidence, or behaviour.

Prompting changes the request, not the model weights

The same clinical case can receive different instructions

- 1 Summarise the report.
- 2 Summarise the report for a pulmonologist.
- 3 Propose differential diagnoses and cite the evidence for each.

Changing the instruction, demonstrations, or output format can substantially change the response.

Advantages

- Immediate and inexpensive
- No training data or weight update

Limitations

- Sensitive to wording and examples
- Cannot reliably supply missing knowledge
- Does not persistently change the model

RAG adds biomedical evidence at inference time

Information passed between components

1. The retriever uses the question as a search query.
2. It returns passages from biomedical sources.
3. The model receives the question and those passages.
4. It generates an answer, ideally with source attribution.

model input = question + retrieved passages

Useful when knowledge is

- current;
- patient specific;
- traceable to a source.

Failure modes

- Relevant evidence is not retrieved.
- Retrieved documents conflict.
- The model ignores or misuses the evidence.

Sources can include articles, guidelines, patient records, protocols, or drug databases.

When should we update the model itself?

Fine-tune when the behaviour must persist

- Learn a repeated input-to-output mapping.
- Enforce a specialized output format or style.
- Improve performance on a stable, well-defined task.
- Adapt representations to domain data.

Fine-tuning does not automatically provide

- current knowledge;
- factual correctness;
- calibrated uncertainty;
- clinical safety.

Use prompting or retrieval for request-specific context or missing evidence rather than persistent behaviour.

Full fine-tuning has a large memory cost

Training must store

- Model parameters
- Gradients
- Optimizer states: running statistics used to compute updates
- Intermediate activations

A 7-billion-parameter example

- FP16 weights alone: roughly 14 GB
- Gradients require additional memory
- Adam stores first- and second-moment estimates for each trainable parameter
- Activations grow with batch size and sequence length

Training requires substantially more memory than inference.

PEFT updates only a small parameter set

Parameter-efficient fine-tuning (PEFT) freezes most or all pretrained weights and learns a much smaller task-specific parameter set.

Method	Trainable part	Core idea
Adapters	Modules inserted between layers	Transform representations through a bottleneck
Prompt tuning	Learned input embeddings	Add trainable soft prompt vectors
Prefix tuning	Learned layer-wise key/value prefixes	Add virtual context to attention
LoRA	Low-rank matrix updates	Modify a linear layer without updating its full matrix

PEFT is a family of mechanisms; the next slides focus on LoRA.

LoRA adds a learned update to frozen weights

$$W' = W + \Delta W, \quad y = W'x = Wx + \Delta Wx$$

Pretrained transformation W

- Has shape $d \times k$.
- Remains frozen during LoRA training.
- Continues to produce the base output Wx .

Task-specific correction ΔW

- Has the same effective shape as W .
- Uses a small trainable parameterization.
- Changes the layer output without replacing W .

The next question is how to represent ΔW with far fewer than dk trainable values.

Low rank creates a small update bottleneck

$$x \in \mathbb{R}^k \xrightarrow{A} u = Ax \in \mathbb{R}^r \xrightarrow{B} Bu \in \mathbb{R}^d$$

$$\Delta W = BA, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k}$$

$$r \ll \min(d, k)$$

The vector u contains only r task-specific coordinates. Matrix B expands them back to a d -dimensional output.

The base transformation Wx remains full dimensional; only the correction is rank constrained.

LoRA assumes the useful weight update lies in a much lower-dimensional subspace than the full matrix.

How is the LoRA update scaled and initialized?

Scaled update

$$\Delta W = \frac{\alpha}{r} BA$$

$$y = Wx + \frac{\alpha}{r} BAx$$

- r : LoRA rank
- α : adaptation magnitude
- α/r : stabilizes scale across ranks

Training and initialization

- W remains frozen.
- Only A and B receive gradients.
- Commonly, one matrix starts random and the other starts at zero.
- Therefore $\Delta W = 0$ initially, so behaviour starts from the pretrained model.

Why does LoRA train so few parameters?

Full fine-tuning

$$W \in \mathbb{R}^{d \times k}$$

$$\text{trainable parameters} = dk$$

Example: $d = k = 4096$, $r = 16$

$$4096^2 = 16,777,216$$

$$16(4096 + 4096) = 131,072$$

LoRA

$$\text{trainable parameters} = dr + rk = r(d + k)$$

$$\frac{131,072}{16,777,216} \approx 0.0078$$

LoRA trains less than 1% as many parameters for this matrix. It does not make the base model small; it makes the trainable update small.

LoRA can target selected projections

Common targets

- Query projection W_Q
- Key projection W_K
- Value projection W_V
- Attention output projection W_O
- Feed-forward projections

Attention reminder

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

LoRA on the query projection

$$Q = X(W_Q + B_Q A_Q)$$

Adapting more layers or matrix types increases capacity, trainable parameters, memory use, and training time.

Which choices control LoRA behaviour?

Rank r

- Small: fewer parameters, less capacity
- Large: more parameters, more capacity

Scaling α

Controls update magnitude.

LoRA dropout

Randomly zeros a fraction of the LoRA-branch input during training. This regularizes the learned update and is disabled at inference.

Target modules

Select which weight matrices receive adapters.

Performance also depends on

- Adapted layers and matrices
- Learning rate
- Training-data quality
- Training objective

How can LoRA be used during inference?

$$y = Wx + BAx$$

Keep adapters separate

base output Wx + adapter output BAx

- One base model supports multiple tasks.
- Small adapters can be switched dynamically.

LoRA makes the adaptation small; it does not compress the base model.

Merge the update

$$W' = W + BA, \quad y = W'x$$

- No separate LoRA branch.
- Inference uses one multiplication with W' .

Why is the frozen base model still large?

$$\text{weight memory in bytes} = \frac{N \times \text{bits per weight}}{8}$$

N is the number of stored weights; for a 7B model, $N \approx 7 \times 10^9$.

Representation	7B-model weight memory
FP32	28 GB
FP16/BF16	14 GB
INT8	7 GB
INT4	3.5 GB

Example: 7×10^9 FP16 weights use $7 \times 10^9 \times 16/8 \approx 14$ GB. These values describe weights only; runtime memory can be larger.

Quantization stores the nearest discrete level

Map a weight to an integer code

$$q = \text{round}\left(\frac{w}{s}\right) + z$$

Reconstruct it for computation

$$\hat{w} = s(q - z)$$

The scale s is the real-valued distance between adjacent codes. The zero point z is the code representing real zero.

Quantization stores q compactly while accepting a controlled reconstruction error.

Example with $s = 0.1, z = 0$

For $w = 0.73$:

$$q = \text{round}(7.3) = 7$$

$$\hat{w} = 0.1 \times 7 = 0.70$$

$$e = w - \hat{w} = 0.03$$

What trade-off does quantization introduce?

$$e = w - \hat{w}$$

Format	Possible codes	Weight memory	Potential error
8 bit	$2^8 = 256$	Higher	Lower
4 bit	$2^4 = 16$	Half of 8 bit	Higher

Fewer codes mean a coarser approximation unless the quantization scheme adapts well to the weight distribution.

Quality depends on bit width, quantization scheme, calibration data, quantized tensors, and the weight distribution.

Symmetric and asymmetric quantization

Scheme	Mapping and use	Example
Symmetric	$q = \text{round}(w/s)$; levels are centred on zero. Simple and often efficient.	A range near $[-1, 1]$ can use signed codes centred on $q = 0$.
Asymmetric	$q = \text{round}(w/s) + z$; a nonzero z shifts the grid to fit an off-centre range.	For $[-0.3, 1.2]$, 4-bit unsigned codes can use $s = 0.1$, $z = 3$: zero maps to $q = 3$, and 0.8 to $q = 11$.

Use a zero point when shifting the integer grid represents an asymmetric real-valued range more effectively.

Quantization scales can use different granularities

The scale s converts one integer step into a real-valued step. A local scale can fit a local weight range more closely.

Granularity	Scale assignment	Trade-off
Per tensor	One scale for the complete tensor	Simple and fast, but potentially less accurate
Per channel	One scale for each output channel	Adapts to different ranges; stores more scales
Group-wise	One scale for each weight group	Common in LLMs; balances accuracy and efficiency

Quantization is not only a choice between 4-bit and 8-bit storage; the grouping strategy also matters.

When should quantization be introduced?

Approach	When quantization enters	Trade-off
Post-training quantization	After model training; weights are converted, sometimes using calibration data	Simple and inexpensive, but accuracy may drop
Quantization-aware training	During training; the forward pass simulates quantization and gradients update high-precision parameters	Can learn robustness to quantization error, but requires additional training

Post-training quantization changes a completed model; quantization-aware training exposes the model to approximation error while it can still adapt.

What exactly can be quantized?

Possible targets

- Model weights
- Activations
- Key-value cache
- Gradients
- Optimizer states

A common LLM deployment

- Stores weights in 4-bit form.
- Dequantizes weights for computation.
- Uses FP16 or BF16 accumulation for matrix multiplication.

A “4-bit model” does not necessarily perform every operation using 4-bit arithmetic.

QLoRA combines quantization with LoRA

LoRA reduces

- The number of trainable parameters
- Gradient and optimizer-state memory

Quantization reduces

- Memory for the frozen base model
- The hardware needed to hold the model

$$\hat{W} = D(Q(W)) \quad y = \hat{W}x + BAx$$

- Q stores the base weights in low precision.
- D dequantizes them for computation.
- The quantized base model remains frozen.
- LoRA adapters use FP16 or BF16.
- Gradients flow only to the adapters.

QLoRA = a quantized frozen base model + trainable LoRA adapters + higher-precision computation.

QLoRA quantizes the base once and trains only adapters

$$y = D(Q(W))x + BAx$$

Stage	Operation	What changes
Before training	Quantize W ; store codes, scales, and metadata	Base representation created once
Forward pass	Dequantize W for the base product; add BAx	Activations only
Optimizer step	Backpropagate through both paths; update only A and B	LoRA adapters

Because W is frozen, it is not updated and re-quantized after each optimizer step.

Training data and loss determine the learned behaviour

Training goal	Data	Objective and learned behaviour
Continued pretraining	Unlabelled domain text	Next-token loss; adapts continuation patterns to the domain
Task supervision	Labelled inputs and outputs	Classification or sequence loss; learns one mapping
Instruction tuning	Instructions and desired responses	Response next-token loss; learns to follow requests

LoRA, QLoRA, and full fine-tuning can all optimize these objectives; the data and loss specify the desired behaviour.

Instruction tuning keeps the next-token loss

Pretraining objective

$$\mathcal{L}_{\text{pre}} = - \sum_t \log p_{\theta}(x_t | x_{<t})$$

- The sequence is raw text.
- Every observed next token can provide a target.
- The model learns broad continuation patterns.

Instruction-tuning objective

$$\mathcal{L}_{\text{inst}} = - \sum_t \log p_{\theta}(y_t | x, y_{<t})$$

- The input x contains an instruction and optional context.
- The desired response y supplies the target tokens.
- Loss is commonly applied to response tokens rather than the instruction.

Two independent choices remain: the instruction-response data specify what behaviour to learn; full fine-tuning, LoRA, or QLoRA specify how the parameters are updated and stored.

How should we choose an adaptation strategy?

First identify the behavioural bottleneck

- **Request or supplied context?**
Use prompting.
- **Missing current or external evidence?**
Add retrieval-augmented generation.
- **Persistent behaviour change?**
Fine-tune for continuation, classification, generation, or instruction following.

Then match the update to the resource bottleneck

- **Compute and memory available?**
Use full fine-tuning.
- **Trainable-parameter memory?**
Use PEFT; use LoRA for a low-rank update.
- **Frozen-model memory?**
Quantize for storage; use QLoRA when adapting.

Efficiency methods determine how we adapt; the training data and loss determine what behaviour is learned.

Adaptation is only the beginning

We now understand how to adapt and compress a foundation model.

The next question is not only whether it is trainable, but whether it is trustworthy.

Evaluation must establish

- Evidence for performance and generalization
- Known limitations and failure modes

Deployment must account for

- Human and tool interaction
- Monitoring after release

Efficient adaptation makes a system feasible; rigorous evaluation determines whether it is fit for use.

Read methodological papers in three passes

With six or more papers each week, extract the method without reading every implementation detail.

1. Triage all papers

Read: title, abstract, method figure, and conclusion.

Leave with: the problem and central method idea.

2. Reconstruct relevant methods

Inspect: inputs and outputs, changed component, training objective, and baseline.

Leave with: a two- or three-step method sketch and its key difference.

3. Inspect selectively

Inspect: key experiments, ablations, and limitations; read technical details only when needed.

Leave with: when the method should work and its main trade-off.

Allocate: Pass 1 for all; Pass 2 for relevant papers; Pass 3 only when needed.
Bring: problem; method sketch; key difference; best result; limitation or question.

Adapted from S. Keshav, [How to Read a Paper](#).

Suggested reading by topic

Transformer and language models

1. Vaswani et al.

Attention Is All You Need

2. Brown et al.

Language Models are Few-Shot Learners

Adaptation and efficient fine-tuning

3. Lewis et al.

Retrieval-Augmented Generation

4. Chung et al.

Scaling Instruction-Finetuned Language Models

5. Hu et al.

LoRA

6. Dettmers et al.

QLoRA

Biomedical use and evaluation

7. Singhal et al.

Large Language Models Encode Clinical Knowledge

All readings are optional for Lecture 1.

Use the three-pass routine to decide which papers deserve closer inspection.